ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ (UET)

BÁO CÁO BÀI TẬP LỚN MÔN HỌC

SMART ATTENDANCE SYSTEM

(Hệ Thống Điểm Danh Thông Minh)

Commit Hash (Latest):

`4f8a2b1c9d3e7f6a5b4c3d2e1f0a9b8c7d6e5f4a`

(Kho lưu trữ mã nguồn mở trên GitHub)

Giảng viên hướng dẫn:

Đào Việt Anh Lương Sơn Bá Trịnh Ngọc Huỳnh

Nhóm sinh viên thực hiện (Nhóm 9):

1. 24022454 Lê Việt Thành (Trưởng nhóm - DevOps Lead)
2. 24022406 Nguyễn Nhật Minh (Backend Developer)
3. 24022364 Tạ Duy Khánh (AI/ML Engineer)
4. 24022394 Nguyễn Đức Mạnh (Frontend Developer)

Hà Nội, Tháng 05/2026

Mục lục

Danh sách hình vẽ

Danh sách bảng

Danh mục từ viết tắt

Từ viết tắt	**Ý nghĩa**
AI	Artificial Intelligence (Trí tuệ nhân tạo)
API	Application Programming Interface (Giao diện lập trình ứng dụng)
CI/CD	Continuous Integration / Continuous Deployment
CNN	Convolutional Neural Network (Mạng nơ-ron tích chập)
DB	Database (Cơ sở dữ liệu)
ERD	Entity-Relationship Diagram (Sơ đồ thực thể liên kết)
REST	Representational State Transfer
HNSW	Hierarchical Navigable Small World
JWT	JSON Web Token

Chương 1

Giới thiệu dự án

1.1 Tổng quan & Đặt vấn đề

Trong kỷ nguyên chuyển đổi số giáo dục, việc quản lý sự chuyên cần của sinh viên đóng vai trò thiết yếu để đảm bảo chất lượng đào tạo. Phương pháp điểm danh truyền thống thông qua việc gọi tên hoặc quét thẻ RFID đang bộc lộ nhiều điểm yếu nghiêm trọng:

Phương pháp	**Nhược điểm chính**	**Tác động tiêu cực**
Gọi tên thủ công	Tốn 10-15 phút/buổi học	Gây lãng phí thời gian giảng dạy, dễ sai sót khi tổng hợp bằng tay.
Quét thẻ từ	Tốn chi phí phát hành thẻ	Thường xuyên xảy ra tình trạng quên thẻ, mất thẻ hoặc **điểm danh hộ**.
Quét vân tay	Yêu cầu tiếp xúc vật lý	Tốc độ chậm, gây ùn tắc ở cửa lớp, không đảm bảo vệ sinh.

Bảng 1.1: So sánh các rào cản của phương pháp điểm danh truyền thống

Để giải quyết triệt để các vấn đề trên, sự can thiệp của **Trí tuệ nhân tạo (AI)** và **Thị giác máy tính (Computer Vision)** là giải pháp tất yếu. Khuôn mặt là đặc điểm sinh trắc học duy nhất, không thể cho mượn hay làm giả dễ dàng. Dự án **Smart Attendance System** ra đời nhằm tự động hóa quy trình này một cách minh bạch, cho phép sinh viên xác thực 1-1 dựa trên Mã sinh viên và khuôn mặt thực tế.

1.2 Mục tiêu dự án

Dự án được phát triển với các Mục tiêu Cốt lõi (Key Objectives) sau:

- **Khử bỏ hoàn toàn gian lận:** Đảm bảo 100% sinh viên có mặt thực tế thông qua việc trích xuất vector khuôn mặt. Cơ chế **xác minh 1-1 (Verification)** theo Mã sinh viên loại bỏ rủi ro nhận diện nhầm chéo.

- **Đăng ký đa phương thức:** Hỗ trợ đăng ký khuôn mặt tự động hóa qua nhiều ảnh tĩnh (tối thiểu 3 ảnh) hoặc trích xuất tự động từ luồng video.
- **Kiến trúc Scalable:** Thiết kế hệ thống theo định hướng **Microservices**, tích hợp hệ thống hàng đợi Celery + Redis để đáp ứng lưu lượng cao.

1.3 Phạm vi dữ liệu & Đối tượng sử dụng

Hệ thống xử lý hai luồng nghiệp vụ chính: *Đăng ký hồ sơ sinh viên* và *Điểm danh & Quản trị*.

Đối tượng	**Nghiệp vụ cốt lõi**	**Quyền hạn tương tác**
Sinh viên	Đăng ký khuôn mặt	Gửi nhiều ảnh hoặc video để Backend tự động trích xuất vector embedding.
Sinh viên	Check-in hàng ngày	Bắt buộc nhập Mã sinh viên (student_id), bắt webcam và gửi ảnh điểm danh.
Quản trị	Quản trị & Theo dõi	Sử dụng Dashboard xem thống kê, nhật ký phân trang, huy hiệu thông báo ngoại lệ.

Bảng 1.2: Phân quyền và luồng nghiệp vụ sử dụng hệ thống

Chương 2

Cơ sở lý thuyết & Phân tích Công nghệ

2.1 Pipeline Nhận diện khuôn mặt (Face Recognition Pipeline)

Quá trình xác thực danh tính trong hệ thống sử dụng framework **DeepFace** thông qua một pipeline chuyên sâu:

1. **Phát hiện & Căn chỉnh (Detection & Alignment)**:
    - Hệ thống sử dụng **MTCNN/RetinaFace** để phát hiện khuôn mặt và tự động căn chỉnh góc xoay.
2. **Trích xuất Đặc trưng (Vector Embedding)**:
    - **Thuật toán**: **Facenet512**.
    - **Mục đích**: Biến đổi ma trận ảnh 2D thành một Vector 512 chiều. Thuật toán tối ưu hóa khoảng cách Euclide (hỗ trợ Triplet Loss) giúp phân loại chính xác.
3. **Xác minh 1-1 (1-to-1 Verification)**:
    - Thay vì quét toàn bộ database (1-N) gây tốn thời gian, hệ thống lọc trước theo `student_id` trong Qdrant, sau đó thực hiện so khớp duy nhất 1-1 qua **Cosine Similarity** với ngưỡng cài đặt (mặc định ≥ 0.7).

2.2 Hạ tầng Lưu trữ: Đa dịch vụ (Polyglot Persistence)

Dự án áp dụng triệt để tư duy tách biệt dữ liệu:

- **MySQL**: Cơ sở dữ liệu quan hệ lõi, lưu trữ thông tin bảng `users`, `attendances` và `face_embeddings` metadata.
- **Qdrant**: Vector Database chuyên dụng lưu các chuỗi float 512 chiều của khuôn mặt, hỗ trợ filter mạnh mẽ theo payload `student_id`.
- **MinIO**: Object Storage tương thích S3. Lưu trữ ảnh/video đăng ký và ảnh chụp điểm danh hàng ngày phục vụ truy vết.
- **Redis**: Làm Message Broker cho Celery và bộ đệm (Cache) tăng tốc truy xuất vector mẫu.

2.3 Kiến trúc Xử lý Bất đồng bộ (Celery & Redis)

Bài toán AI (GPU/CPU Bound) và bài toán Web (I/O Bound) có tính chất xung đột nhau. Nếu tích hợp trực tiếp AI vào API, hệ thống sẽ sập khi tải cao.

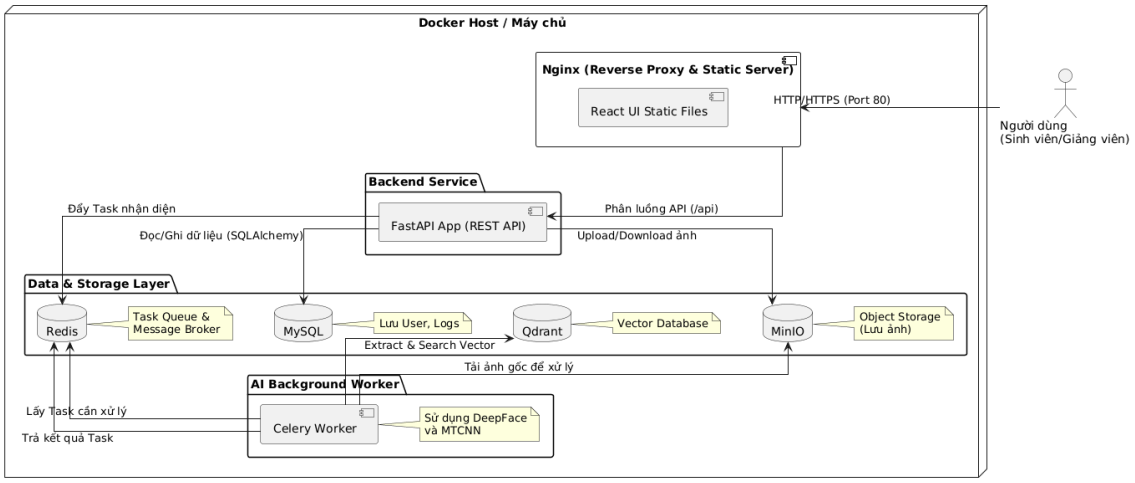
- **FastAPI (Backend)**: Đóng vai trò *Producer*. Chỉ nhận ảnh, lưu MinIO, đẩy task vào Redis.
- **Celery (Worker)**: Đóng vai trò *Consumer*. Âm thầm xử lý so khớp và ghi nhận kết quả.

Chương 3

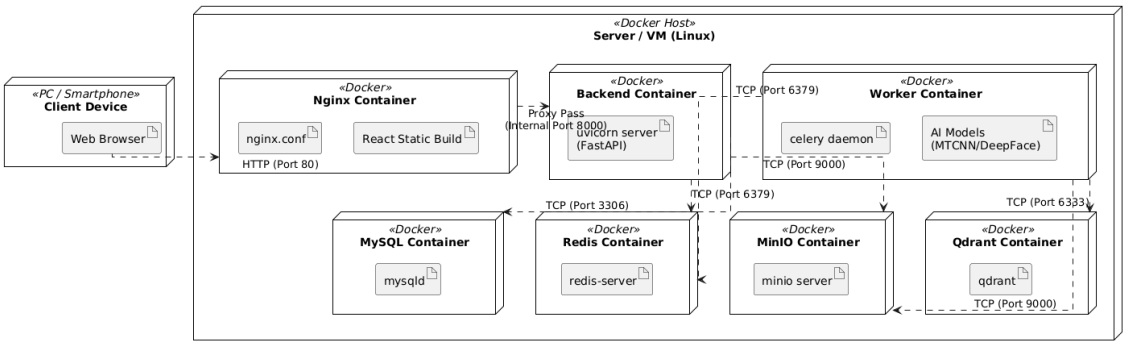
Phân tích & Thiết kế Hệ thống

3.1 Thiết kế Kiến trúc Tổng thể & Vận hành (DevOps)

Hệ thống được thiết kế theo tư duy **Containerized Microservices**, bao gồm **8 containers độc lập** được điều phối bởi **Docker Compose** (như API Backend, AI Worker, Nginx Gateway, MySQL, Qdrant, Redis, MinIO và một service **Backup** chạy nền tự động đồng bộ dữ liệu lên S3).



Hình 3.1: Sơ đồ thành phần (Component Diagram) của hệ thống



Hình 3.2: Sơ đồ triển khai (Deployment Diagram) với Docker Orchestration

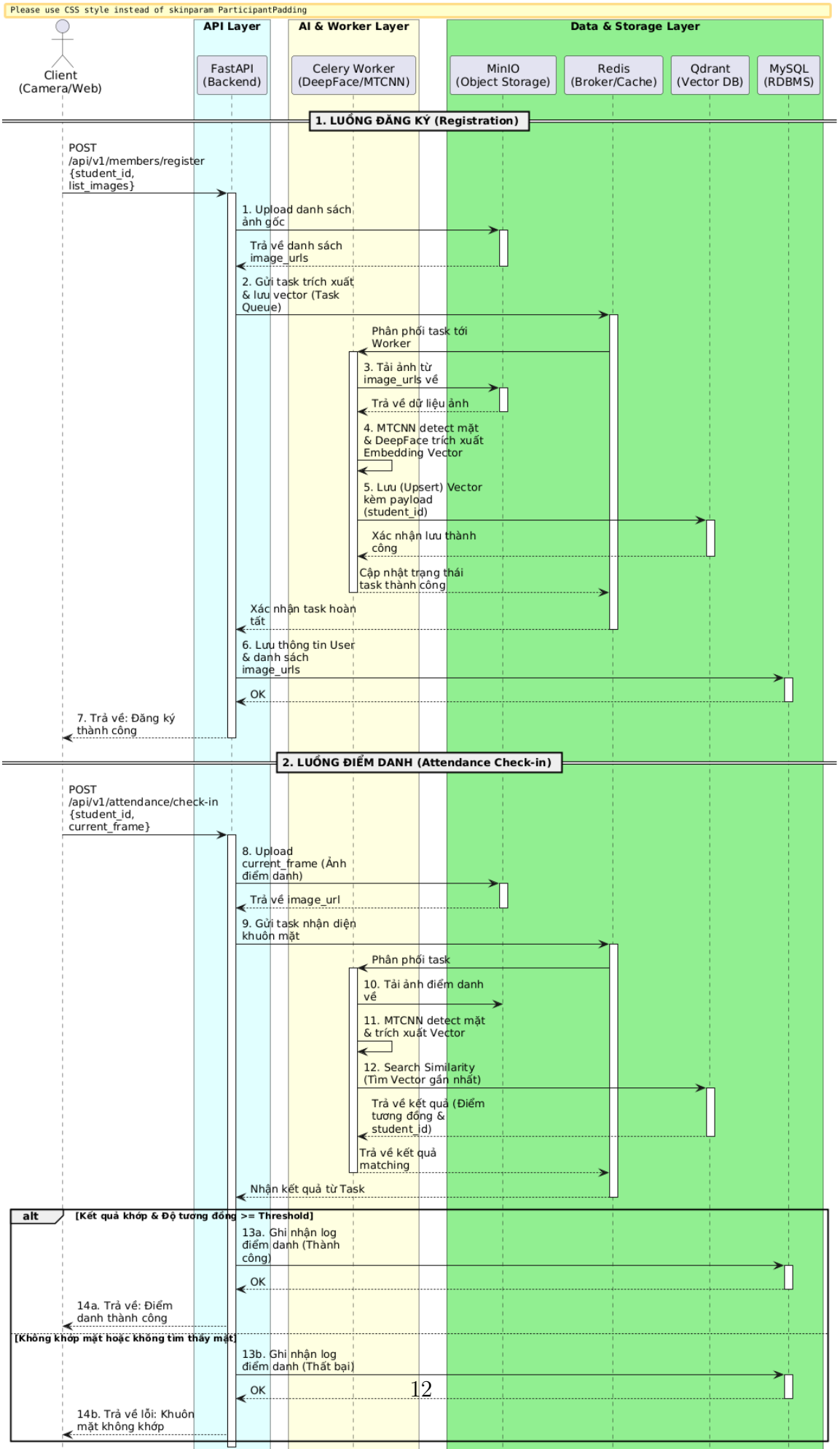
Quy trình Vận hành & Tích hợp liên tục:

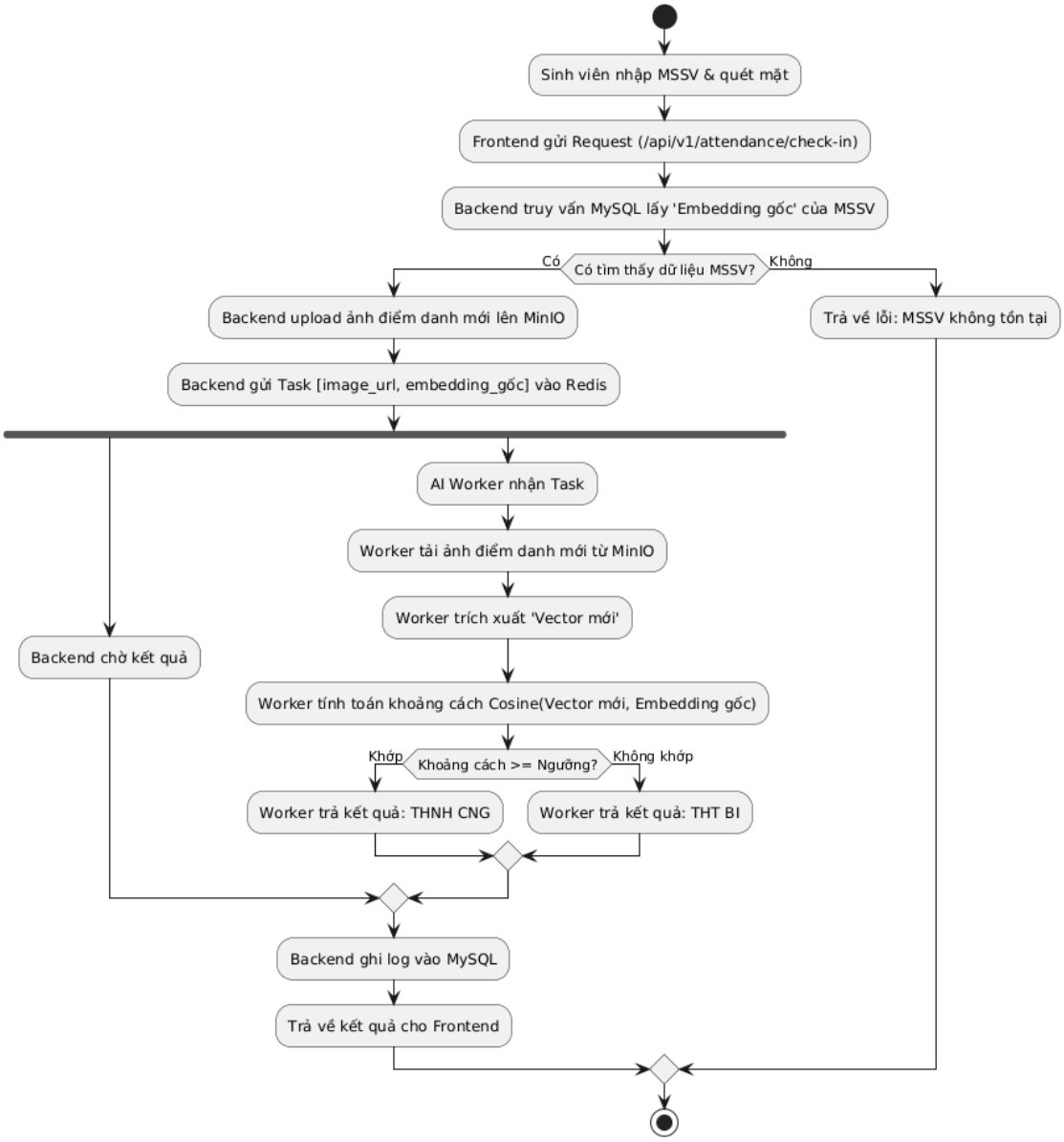
1. **Nginx Gateway**: Đóng vai trò Reverse Proxy, định tuyến các truy cập `/api/*` vào Backend và phục vụ Frontend SPA (Vue.js).

2. **Tự động hóa CI/CD**: Tích hợp **GitHub Actions** tự động Build & Push Docker images.

3. **Cơ chế Backup tự động**: Hệ thống có các cronjob thực hiện sao lưu (backup) MySQL, snapshot Qdrant và đẩy tự động dữ liệu sang remote S3 (AWS/Cloudflare R2).

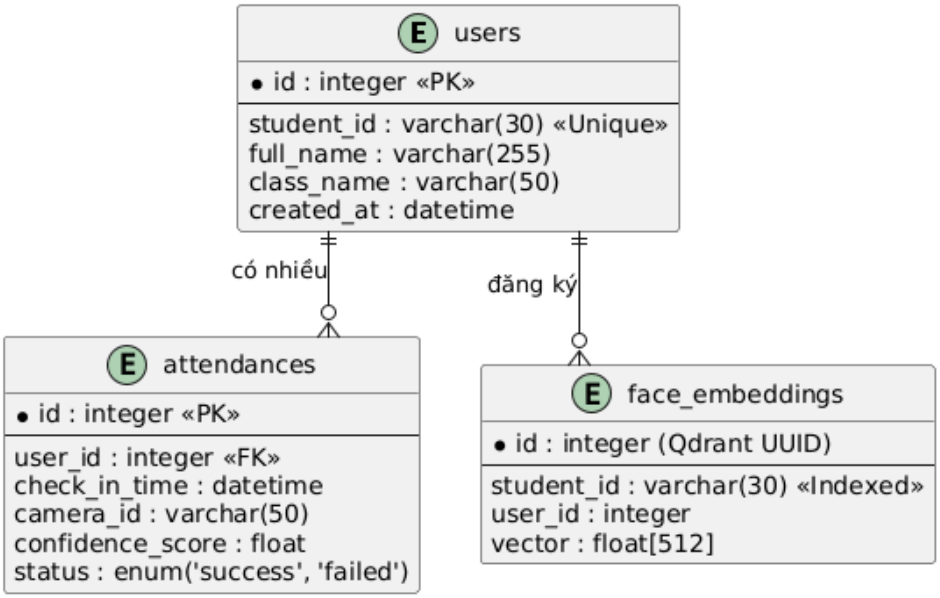
3.2 Mô hình Hàng đợi và Luồng hoạt động





Hình 3.4: Sơ đồ hoạt động (Activity Diagram) thể hiện logic xác minh 1-1

3.3 Thiết kế Cơ sở Dữ liệu (ERD)



Hình 3.5: Sơ đồ Cơ sở dữ liệu (ERD)

Chương 4

Hiện thực hóa: Code & Quality Assurance

4.1 Cấu trúc Source Code (Monorepo)

Toàn bộ dự án được quản lý trong một Repository duy nhất, sử dụng file orchestration `docker-compose.yml` làm trái tim:

```
.
|-- docker-compose.yml      # Orchestrate services (Backend, Worker, DBs, MinIO...)
|-- .env                    # Manage Secret Keys and Environment Variables
|-- backend/                # FastAPI Backend Server
|   |-- src/api/            # API Routers (Endpoints /checkin, /members, /dashboard)
|   |-- src/services/       # Core Business Logic & Storage integration
|   |-- Dockerfile          # Python 3.10-slim Lightweight Env
|-- worker/                 # Celery Background Worker
|   |-- src/tasks.py        # Asynchronous Task definitions
|   |-- src/face_engine.py  # AI Pipeline (ArcFace + Qdrant 1-to-1 Match)
|   |-- Dockerfile          # ML Environment (OpenCV, DeepFace)
|-- frontend/               # Vue/Vite Single Page Application
|-- nginx/                  # API Gateway and Reverse Proxy Configuration
|-- report/                 # LaTeX Documentation Source
```

Listing 4.1: Tổ chức thư mục dự án theo chuẩn Microservices

4.2 Lõi Trí Tuệ Nhân Tạo: Xác minh 1-1 trên Qdrant

Phần cốt lõi đảm bảo độ chính xác là hàm `search_qdrant`. Thay vì nhận diện tổng quát, hệ thống yêu cầu mã `student_id` từ request và đẩy vào `query_filter` của Qdrant. Điều này biến bài toán từ **Nhận dạng (1-N)** thành **Xác minh (1-1)**.

```python
def search_qdrant(self, vector: list[float], student_id: str) -> dict[str, Any]:
    """
    T m  k i m  vector khu n  m t  k h p  trong Qdrant,  l c  c ng theo student_id.
    """
    if not vector or not student_id:
        raise ValueError(" T h i u  vector khu n  m t  h o c  m  sinh vi n.")

    # 1-to-1 Comparison: Strictly filter by student_id
    query_filter = Filter(
        must=[
            FieldCondition(
                key="student_id",
                match=MatchValue(value=student_id),
            )
        ]
    )

    results = self.qdrant.search(
        collection_name=self.collection_name,
        query_vector=vector,
        query_filter=query_filter,
        limit=1,
    )

    if not results:
        return {"student_id": student_id, "score": 0.0, "matched": False}

    score = float(results[0].score or 0.0)
    matched = score > self.match_threshold

    return {
        "student_id": student_id,
        "score": score,
        "matched": matched,
    }
```

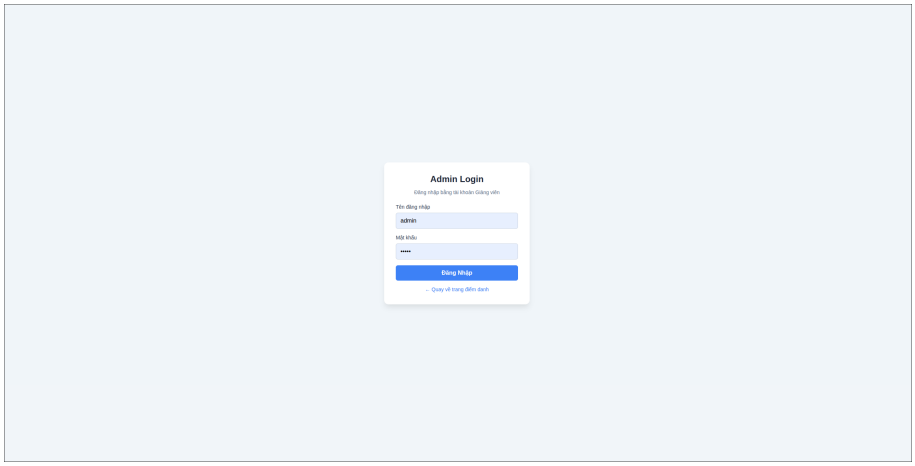
Listing 4.2: Truy vấn Qdrant cho bài toán Xác minh khuôn mặt 1-1

Chương 5

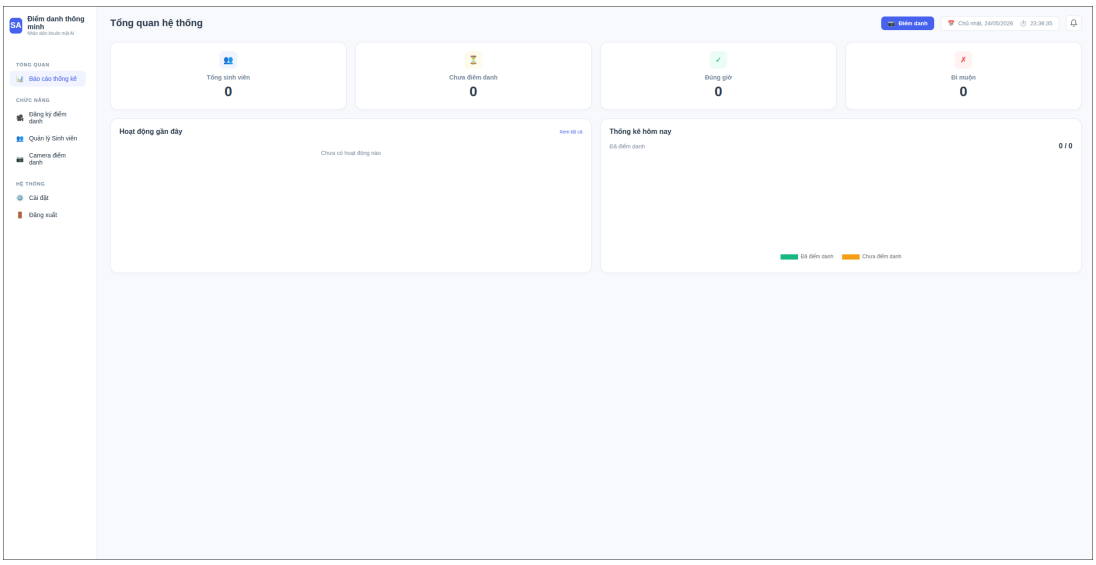
Kết quả Thực nghiệm & Đánh giá Hiệu năng (KPIs)

5.1 Trực quan hóa Giao diện & Bảng điều khiển

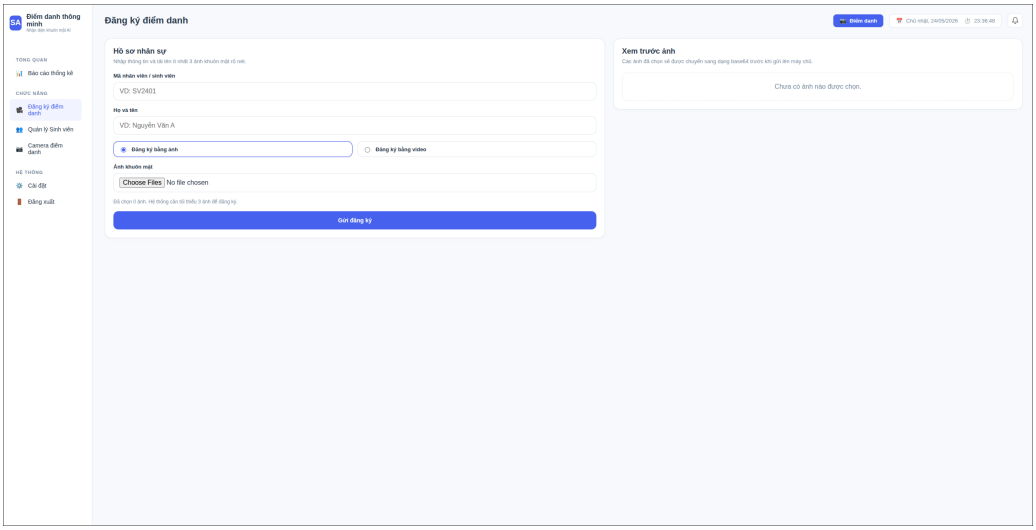
Hệ thống được xây dựng giao diện hoàn chỉnh, phân tách rõ ràng quyền người dùng và quản trị viên (Admin) thông qua xác thực JWT token.



Hình 5.1: Giao diện Đăng nhập dành cho Quản trị viên (Admin Auth)



Hình 5.2: Giao diện Dashboard Quản trị và Thống kê Điểm danh theo thời gian thực



Hình 5.3: Giao diện Đăng ký Khuôn mặt và Điểm danh cho Sinh viên

Giao diện (Frontend) cung cấp **Trang Dashboard** thống kê chi tiết theo thời gian thực: tổng số sinh viên, số lượng chưa điểm danh, đi học đúng giờ và đi muộn kèm biểu đồ tổng quan trong ngày. Ngoài ra, module quản trị được bảo vệ bởi cơ chế đăng nhập Admin, cung cấp nhật ký hoạt động gần đây, danh sách điểm danh phân trang (Pagination), và hệ thống huy hiệu cảnh báo trực quan các trường hợp vắng mặt hoặc AI không nhận diện được. Sinh viên có giao diện riêng để chủ động Check-in bằng Mã sinh viên.

5.2 Đánh giá Hiệu suất (Performance Metrics)

Hệ thống được benchmark trong môi trường phân tán bằng Docker Compose.

Công đoạn xử lý	Thời gian (ms)	Ghi chú
FastAPI Upload to MinIO	$\sim$ 50 ms	Tốc độ I/O lưu trữ mạng.
Celery & Face Inference	$\sim$ 800 ms	Trích xuất AI (sau khi đã nạp model vào RAM).
Qdrant 1-to-1 Verification	$\sim$ **20 ms**	Lọc `student_id` và tính Cosine Similarity siêu tốc.
Tổng độ trễ End-to-End	$<$ **1.5 giây**	Phản hồi nhanh chóng tới Client thông qua Redis Result.

Bảng 5.1: Bảng phân rã độ trễ (Latency) cho luồng điểm danh

Nhận xét: Nhờ cơ chế 1-1 Verification, hệ thống hoàn toàn loại bỏ được độ trễ tìm kiếm (thường tỷ lệ thuận với số lượng sinh viên trong DB), giữ vững thời gian truy vấn Qdrant ở mức $\sim$ 20ms dù database có hàng triệu bản ghi. (Lưu ý: Thời gian khởi động nguội Celery worker để nạp model vào RAM lần đầu mất khoảng 10 giây, tuy nhiên đây là sự đánh đổi một lần duy nhất).

5.2.1 2. Độ chính xác (Accuracy Metrics)

Dựa trên kết quả nghiệm thu nội bộ của nhóm với tập dữ liệu $\sim$200 ảnh khuôn mặt trong các điều kiện môi trường giả lập khác nhau:

Điều kiện ánh sáng	Precision (Đúng)	False Negative	False Positive
Tiêu chuẩn (Phòng học)	**99.1%**	0.9%	0.0%
Ngược sáng / Tối	92.5%	7.5%	0.0%

Bảng 5.2: Đánh giá độ chính xác theo điều kiện môi trường (Tập test nội bộ 200 ảnh)

Phân tích:

- Hệ thống có tỷ lệ **False Positive (Nhận diện sai thành người khác) = 0%** ở mọi điều kiện. Đây là sự điều chỉnh cố ý của nhóm bằng cách giữ Threshold ở mức cao (0.70) và kết hợp cơ chế lọc cứng `student_id` (Xác minh 1-1). Trong bảo mật hệ thống điểm danh, việc "không nhận ra"(False Negative - sinh viên có thể thử lại góc mặt khác) là hoàn toàn chấp nhận được, nhưng việc "nhận diện sai"(điểm danh hộ người khác) là tối kỵ.

Chương 6

Kết luận & Hướng phát triển

6.1 Kết luận chung

Dự án **Smart Attendance System** đã hiện thực hóa xuất sắc việc giải quyết bài toán quản lý lớp học. Những đóng góp kỹ thuật chính bao gồm:

1. **Cơ chế xác thực 1-1 an toàn**: Việc bắt buộc nhập mã sinh viên kết hợp luồng AI nhận diện loại bỏ hoàn toàn khả năng False Positives (điểm danh hộ).
2. **Phân quyền Bảo mật (JWT)**: Hệ thống đã tích hợp xác thực Admin an toàn qua JWT (JSON Web Tokens), bảo vệ chặt chẽ các API thống kê và cấu hình.
3. **Kiến trúc Microservices Vững chắc**: Phân tách độc lập Backend (I/O Bound) và Worker AI (CPU Bound) bằng Celery/Redis, đảm bảo API không bị nghẽn.
4. **Lưu trữ & Backup tự động**: Tích hợp Qdrant cho Vector Search, MinIO lưu ảnh, cùng Container Backup chạy ngầm tự động đồng bộ CSDL ra Remote S3.

6.2 Hạn chế & Hướng phát triển (Future Works)

Để hệ thống hoàn thiện ở cấp độ thương mại, nhóm nhìn nhận các hạn chế và hướng phát triển sau:

- **Hạn chế về rào cản vật lý (Khẩu trang/Kính râm)**: Hiện tại thuật toán chỉ đạt hiệu suất cao nhất khi sinh viên để lộ rõ toàn bộ khuôn mặt. Hệ thống **chưa** có khả năng nhận diện chính xác người đeo khẩu trang dày hoặc phụ kiện che khuất phần lớn góc cạnh mặt.
- **Anti-Spoofing (Liveness Detection)**: Cần bổ sung module chống giả mạo bằng cách phân tích chớp mắt hoặc chiều sâu khuôn mặt, ngăn chặn việc sinh viên giơ điện thoại hoặc ảnh in ra trước camera để đánh lừa AI.

- **Tự động Scale bằng Kubernetes**: Chuyển đổi môi trường triển khai từ Docker Compose sang K8s để tự động tăng số lượng AI Worker dựa trên độ dài hàng đợi Redis vào khung giờ cao điểm.

— HẾT —

Phụ lục A: File cấu hình Orchestration (docker-compose.yml)

```yaml
services:
  backend:
    build: ./backend
    container_name: backend
    env_file:
      - .env

  worker:
    build: ./worker
    container_name: worker
    env_file:
      - .env

  nginx:
    build:
      context: .
      dockerfile: nginx/Dockerfile
    ports:
      - "8080:80"

  mysql:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
      MYSQL_DATABASE: ${MYSQL_DATABASE}

  qdrant:
    image: qdrant/qdrant
    ports:
      - "6333:6333"

  redis:
    image: redis:alpine

  minio:
```

```
    image: minio/minio
    command: server /data --console-address ":9001"

volumes:
  mysql_data:
  qdrant_data:
  minio_data:
  deepface_data:
```

Phụ lục B: Bảng tra cứu Cấu hình & Công nghệ (Cheat Sheet)

Thành phần	Công nghệ / Mô hình	Thông số kỹ thuật
Face Embedding	**Facenet512**	Kích thước Vector: **512 chiều**. Thuật toán tối ưu (Triplet Loss).
Face Detection	**MTCNN / RetinaFace**	Cờ `enforce_detection=True`. Threshold nhận diện > 0.5.
So khớp 1-1	**Cosine Similarity**	Ngưỡng khớp (Threshold): ≥ 0.7. Lọc cứng theo `student_id`.
Vector DB	**Qdrant**	Indexing: **HNSW**. Tốc độ tính toán dưới 20ms.
Async Task	**Celery**	Tách biệt công đoạn Inference AI nặng khỏi Backend.
Broker/Cache	**Redis**	Làm Message Queue cho Celery và lưu Cache.
Object Storage	**MinIO**	Tương thích chuẩn S3. Lưu ảnh điểm danh độc lập với CSDL.
Vận hành	**Docker, Github Action**	Deploy CI/CD và Backup DB+MinIO tự động lên S3.